BETA TEST PLAN – VoidMC

1. Project context

VoidMC is a high-performance, fully customizable Minecraft server framework built entirely from scratch using **Rust**. Unlike end-user products (like a finished game server), VoidMC is a development tool designed for other developers. It eliminates the overhead of legacy Java solutions by providing a strict Entity Component System (ECS) architecture.

The objective of this Beta is to validate the **core engine's API and systems**. We aim to demonstrate that a developer can use the VoidMC framework to effortlessly register systems (AI, Physics), handle networking events, and manage data persistence without interacting with low-level protocol details.

2. User Roles

The following roles will be involved in beta testing.

Role Name	Description
Developer	The primary user. They use the VoidMC crate to register components, define logic, and configure the server runtime (tokio tasks, tick loop).
Player	The end-user used to visually verify that the logic implemented by the Developer via the framework is functioning correctly in the Minecraft client.

3. Feature table

The following features demonstrate the framework's capabilities and will be shown during the defense.

Feature ID	User role	Feature name	Short description
F1	Developer	Network Status Handling	The framework automatically handles TCP handshakes and status (Ping/MOTD) requests
F2	Developer	Player Entity Initialization	The ECS automatically creates and spawns a player entity upon connection
F3	Developer	Chunk Streaming Pipeline	The engine automatically serializes and streams chunks to clients in range

Feature ID	User role	Feature name	Short description
F4	Developer	Position Component Sync	The engine processes movement packets and updates Position components in the ECS
F5	Player	Entity State Broadcasting	The framework synchronizes entity presence and movement between clients
F6	Developer	Chat Event API	The framework exposes a chat event that developers can listen to and broadcast
F7	Player	World Interaction Handling	The engine processes block breaking/placing packets and updates the world state
F8	Developer	World Serialization API	The framework provides an API to save chunk modifications to disk automatically
F9	Developer	Custom Entity Registration	Developers can register non-player entities (mobs) into the ECS
F10	Developer	AI Behavior Injection	Developers can attach AI behaviors to entities which are executed by the tick loop

4. Success Criteria

These metrics validate that the framework performs its tasks correctly and efficiently.

Feature ID	Key success criteria	Indicator/metric	Result
F1	The server responds to status requests with the configured MOTD and version	Latency $<$ 50ms, MOTD visible in client	Achieved
F2	The engine successfully injects the player entity into the ECS world	Player spawns at correct coordinates	Achieved
F3	The chunk management pipeline creates valid packets readable by the client	Render distance 8 chunks loaded $<$ 1s	Achieved
F4	The ECS updates position components at each tick based on network input	Movement is smooth, no console warnings	Achieved
F5	The synchronization logic correctly filters and sends updates to observers	Players see each other moving in real-time	Achieved
F6	The event bus correctly propagates chat packets to all subscribers	Message received by all clients $<$ 200ms	Achieved

Feature ID	Key success criteria	Indicator/metric	Result
F7	The world state is mutated in memory and acknowledged by the client	Block updates visible instantly	Achieved
F8	The IO logic writes modified chunks to the file system upon save trigger	100% of modified blocks persist after restart	Achieved
F9	Registered entities are correctly instantiated with their default components	Entities visible in-game	Achieved
F10	The AI logic updates entity positions/rotation independently of players	Entities wander autonomously	Achieved